

JavaScript Variables

| What is a Variable?

A **variable** is a named container used to store data in memory. It allows you to save information that can be used, updated, or reused later in your program.

Variables can store different types of data, such as:

- Number
- String
- Boolean
- Null
- Undefined
- Object
- Array
- Function

Instead of writing the same value repeatedly, you can store it inside a variable and use it whenever needed.

| Example

```
let name = "Rokon";  
const age = 23;  
  
console.log(name);  
console.log(age);
```

OUTPUT

```
Rokon  
23
```

Why Do We Use Variables?

Variables make programming much easier.

They help us to:

- Store data
 - Reuse values
 - Update values
 - Make code clean and readable
 - Reduce duplicate code
-

| Without Variables

```
console.log("Rokon");  
console.log("Rokon");  
console.log("Rokon");
```

OUTPUT

```
Rokon  
Rokon  
Rokon
```

EXPLANATION

The same value is written multiple times. If you want to change the name later, you must change every occurrence manually.

| With Variables

```
let name = "Rokon";  
  
console.log(name);  
console.log(name);  
console.log(name);
```

OUTPUT

```
Rokon  
Rokon  
Rokon
```

EXPLANATION

The value is stored only once. If you change the variable, every place using that variable automatically gets the updated value.

Variable Naming Rules

A variable name must follow JavaScript rules.

- Must start with a letter, `_`, or `$`
 - Cannot start with a number
 - Cannot contain spaces
 - Is case-sensitive
 - Cannot use JavaScript reserved keywords
-

Valid Variable Names

```
let name = "Rokon";  
let firstName = "Rokon";  
let _age = 23;  
let $price = 100;
```

OUTPUT

No Output

EXPLANATION

These are valid variable names because they follow JavaScript naming rules.

Invalid Variable Names

```
let 1name = "Rokon";  
let first name = "Rokon";  
let let = 10;
```

OUTPUT

SyntaxError

EXPLANATION

- `1name` starts with a number.
 - `first name` contains a space.
 - `let` is a reserved keyword.
-

JavaScript is Case-Sensitive

JavaScript treats uppercase and lowercase letters as different.

```
let name = "Rokon";  
let Name = "Rahim";  
let NAME = "Karim";  
  
console.log(name);  
console.log(Name);  
console.log(NAME);
```

OUTPUT

```
Rokon  
Rahim  
Karim
```

EXPLANATION

These are three different variables.

Variable Declaration

JavaScript provides three keywords to declare variables.

- var
- let
- const

Each behaves differently.

var

Definition

`var` is the old way of declaring variables.

Features:

- Function scoped
 - Can be redeclared
 - Can be reassigned
-

Syntax

```
var variableName = value;
```

Example

```
var name = "Rokon";

var name = "Rahim";

name = "Karim";

console.log(name);
```

OUTPUT

Karim

EXPLANATION

- First declaration → Rokon
- Redeclared → Rahim
- Reassigned → Karim

Final value becomes **Karim**.

Scope Example

```
if (true) {  
  var city = "Dhaka";  
}  
  
console.log(city);
```

OUTPUT

Dhaka

EXPLANATION

Because `var` is function scoped, it ignores block scope.

let

Definition

`let` is the modern way of declaring variables.

Features:

- Block scoped
 - Cannot be redeclared
 - Can be reassigned
-

Syntax

```
let variableName = value;
```

Example

```
let age = 23;  
  
age = 24;  
  
console.log(age);
```

OUTPUT

```
24
```

EXPLANATION

The variable value changes from **23** to **24**.

Redeclaration Error

```
let age = 23;  
  
let age = 24;
```

OUTPUT

SyntaxError

EXPLANATION

A `let` variable cannot be declared twice in the same block.

Scope Example

```
if (true) {  
  let city = "Dhaka";  
}  
  
console.log(city);
```

OUTPUT

ReferenceError

EXPLANATION

`city` exists only inside the block.

const

Definition

`const` creates a constant variable.

Features

- Block scoped
 - Cannot be redeclared
 - Cannot be reassigned
 - Must be initialized
-

Syntax

```
const variableName = value;
```

Example

```
const PI = 3.1416;  
  
console.log(PI);
```

OUTPUT

```
3.1416
```

| Reassignment Error

```
const PI = 3.1416;  
  
PI = 3.14;
```

OUTPUT

```
TypeError
```

EXPLANATION

A constant variable cannot be assigned a new value.

const with Objects

```
const person = {  
  name: "Rokon",  
  age: 23  
};  
  
person.age = 24;  
  
console.log(person);
```

OUTPUT

```
{  
  name: 'Rokon',  
  age: 24  
}
```

EXPLANATION

The object itself is constant, but its properties can still be modified.

const with Arrays

```
const numbers = [10, 20, 30];  
  
numbers.push(40);  
  
console.log(numbers);
```

OUTPUT

```
[10, 20, 30, 40]
```

EXPLANATION

You cannot replace the array, but you can change its elements.

Primitive Data Types

Primitive data types store a single value.

```
let name = "Rokon";  
let age = 23;  
let isStudent = true;  
let salary = null;  
let address;  
  
console.log(name);  
console.log(age);  
console.log(isStudent);  
console.log(salary);  
console.log(address);
```

OUTPUT

```
Rokon  
23  
true  
null  
undefined
```

Non-Primitive Data Types

Non-primitive values can store multiple pieces of information.


```
let person = {  
  name: "Rokon",  
  age: 23  
};  
  
let numbers = [10, 20, 30];  
  
function greet() {  
  console.log("Hello");  
}  
  
console.log(person);  
console.log(numbers);  
greet();
```

OUTPUT

```
{ name: 'Rokon', age: 23 }  
[10,20,30]  
Hello
```

Variable Scope

Scope determines where a variable can be accessed.

JavaScript has three scopes.

- Global Scope
 - Function Scope
 - Block Scope
-

Global Scope

```
let name = "Rokon";

function show() {
  console.log(name);
}

show();
```

OUTPUT

```
Rokon
```

EXPLANATION

Global variables are accessible from anywhere.

Function Scope

```
function test() {  
  var age = 23;  
  
  console.log(age);  
}  
  
test();  
  
console.log(age);
```

OUTPUT

```
23  
ReferenceError
```

EXPLANATION

The variable exists only inside the function.

Block Scope

```
if (true) {  
  let city = "Dhaka";  
  const country = "Bangladesh";  
}  
  
console.log(city);  
console.log(country);
```

OUTPUT

```
ReferenceError  
ReferenceError
```

EXPLANATION

Both variables exist only inside the block.

Hoisting

Hoisting moves declarations to the top before execution.

| var

```
console.log(name);  
  
var name = "Rokon";
```

OUTPUT

```
undefined
```

EXPLANATION

The declaration is hoisted, but the value is assigned later.

| let

```
console.log(age);  
  
let age = 23;
```

OUTPUT

ReferenceError

EXPLANATION

`let` is hoisted but remains inside the Temporal Dead Zone until initialization.

| `const`

```
console.log(PI);  
  
const PI = 3.1416;
```

OUTPUT

ReferenceError

EXPLANATION

`const` also stays inside the Temporal Dead Zone before initialization.

Temporal Dead Zone (TDZ)

The **Temporal Dead Zone (TDZ)** is the period between entering a block and the line where a `let` or `const` variable is initialized.

```
{  
  let age = 23;  
  
  console.log(age);  
}
```

OUTPUT

```
23
```

EXPLANATION

The variable can only be used after its declaration.

Comparison Table

Feature	var	let	const
Scope	Function	Block	Block
Redeclare	Yes	No	No
Reassign	Yes	Yes	No
Hoisted	Yes	Yes (TDZ)	Yes (TDZ)
Must Initialize	No	No	Yes
Modern Use	Rare	Common	Most Preferred

Best Practices

| Use `const` by default

```
const company = "Google";  
  
console.log(company);
```

OUTPUT

```
Google
```

Use `let` when the value changes

```
let score = 0;

score++;

console.log(score);
```

OUTPUT

```
1
```

Avoid using `var`

Modern JavaScript projects rarely use `var` because it can introduce bugs due to function scope and redeclaration.

Summary

- Variables store data in memory.
- JavaScript has three variable keywords: `var`, `let`, and `const`.
- `const` is the safest and most preferred choice.
- Use `let` when the value needs to change.
- Avoid using `var` in modern JavaScript.

- Variable names are case-sensitive.
- Always choose meaningful variable names.
- Understanding scope and hoisting helps you avoid common JavaScript mistakes.